

1 Role, Ownership, and Rubric Coverage

In our four-person team, my contribution was the integration and validation role required for a group of four. I was not the primary owner of authentication, quiz business logic, leaderboard data, or admin CRUD. My work was to make those independently developed parts behave consistently as one MERN application, and to leave evidence that the final system matched the assignment specification.

Rubric area	How my work contributed
Backend Architecture & APIs	Shared Express app behaviour, response envelopes, central error handling, route documentation, and Jest/Supertest coverage.
Security & Validation	Rate-limit responses, protected-route failures, envelope consistency, and Swagger alignment for 401, 403, and 429.
Frontend UI & UX	Initial shared React shell, API client, route wiring, and persisted theme; I did not own the individual feature pages.
Documentation & Process	README/API/Postman/audit evidence so markers can verify the branch and endpoints without guessing.

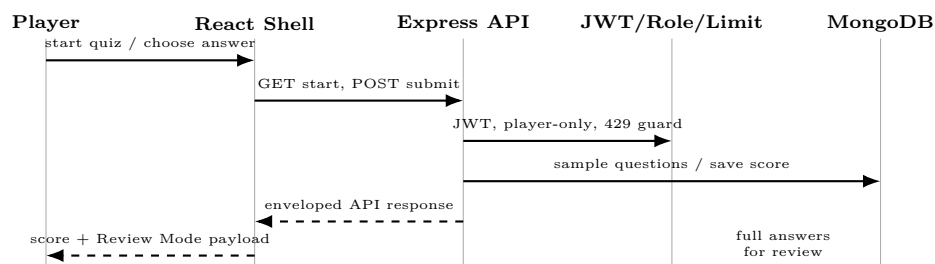
2 Major Technical Challenge

The hardest issue I handled was not a visual bug; it was a contract bug. Review Mode depends on the same answer order being used in four places: the shuffled options returned by `/api/quiz/start`, the visible index submitted by the frontend, the backend scoring logic, and the saved payload later shown in Review Mode. A test still assumed the raw seed answer index, which was unsafe after option shuffling.

I fixed `backend/src/tests/api.test.js` so it follows the real user path: start quiz, locate the correct visible option in the shuffled list, submit that visible index, and assert that the saved review payload matches the same ordering. Commit `bba8b06` is the direct fix. I then tightened the API documentation in `4a8fe22` and `d0d07f6`; the final branch reinforced the same contract through signed attempt tokens and persisted option order in `73d24c6` and `f5d3d98`.

start quiz -> read shuffled options -> submit selected visible index -> assert saved Review Mode payload

3 Subsystem UML Sequence Diagram



4 Reflection on the Approved Variation

Our approved variation was Review Mode after completion. My design view was that Review Mode should not change the core scoring rule: the player still earns +1 per correct answer and no extra scoring scheme is introduced. The extra responsibility is data integrity after the attempt. Because answers are shuffled, the system must preserve enough information to reconstruct what the player saw, selected, and got right or wrong.

In practice, this gave me three integration checks to protect: the start response should not leak `correctAnswer`; submit should score the visible selected index in the shuffled list; and history/review should explain the saved attempt rather than reconstructing a different answer order later. Because our variation is Review Mode, we do not require a pre-quiz category-selection step; topics are supporting metadata for feedback, not the main variation flow.

5 Commit History Analysis

Only commits verified on `origin/final` are used below; old local-only hashes and removed docs are not used as final evidence.

Hash	Focus	Contribution evidence
7a61d82	scaffold	Shared project layout, root README/package scripts, backend/frontend foundations.
6686eae	D tests/docs	Seed support, Postman evidence, response-envelope checks, and backend regression coverage.
972fc77	QA alignment	Reconciled tests and documentation with the delivered API surface.
160ac65	README QA	Flagged leaderboard/auth QA risk instead of leaving it implicit.
bba8b06	shuffle tests	Fixed shuffled visible-option scoring and saved Review Mode payload assertions.
4a8fe22	API docs	Aligned Swagger with protected leaderboard behaviour.
d0d07f6	error docs	Documented player-only and rate-limited quiz responses.
73d24c6	integrity	Added signed attempt tokens, replay protection, persisted option order, and tests.
fd08cc8	boundary	Tightened admin login and player-route separation.
a6e4ad7	robustness	Separated register rate limiting and exposed the API root with backend coverage.
f5d3d98	final lock	Locked shuffled answer-mapping tests before the assessment-ready merge.

The early alias `Tom_Tian` appears on commits such as `6686eae`; later commits use `Tom Tian`.

6 Final Verification and What I Would Defend in Marking

The common theme in my commits is integration ownership: making separately built features testable together. The part I would defend most strongly is the Review Mode contract. If `/api/quiz/start`, `/api/quiz/submit`, saved scores, and review rendering disagree, the project may appear to work while giving incorrect feedback.

The verification evidence I would rely on is: `npm test -prefix backend -- --runInBand`, `npm test -prefix frontend`, `npm run build -prefix frontend`, `python3 -m json.tool docs/postman-collection.json`, `git diff --check`, and a local `npm run demo smoke` test. The single commit I would defend most strongly is `bba8b06` -- the Review Mode test contract.

7 Ownership Boundary and Lessons

The main lesson was that integration work is not the same as taking over other members' features. I could have resolved some cross-owner issues by editing quiz or admin business logic directly, but that would have crossed the ownership boundary the group-of-four integration role defines. Instead, I focused on shared contracts around those features: tests, route documentation, response formats, and verification notes. For future group projects, I would start route-documentation cross-checking earlier because a route can be technically correct while its documented contract is stale.